LEVELD.AI

Technical Challenge Brief

Contract Clause Risk Extractor

Duration	Fee on completion	Deliverable
24 hours	£50	Working web app

1. Overview

We're Leveld.ai — an AI-powered platform that extracts structured intelligence from unstructured business communications. This challenge is designed to assess how you think, design, and build when AI is a core part of the product.

You will build a Contract Clause Risk Extractor: a tool that accepts a contract or statement of work (SOW) in PDF or plain text form, analyses its contents using AI, and surfaces potential risk clauses with severity ratings, plain-English explanations, and actionable recommendations — all through a polished, professional user interface.

Why this challenge?

This is not a trivial CRUD app. It requires you to design a multi-step AI pipeline, make thoughtful decisions about how to present dense, complex information to non-expert users, and build something that feels trustworthy and production-ready — not a hackathon prototype.

2. The Problem

Project managers and commercial leads regularly sign contracts and statements of work without the benefit of legal counsel. Risks buried in dense clause language — around liability caps, payment terms, IP ownership, termination rights, and change control — routinely go unnoticed until it's too late.

Your tool should make it simple for a non-lawyer to upload a contract, instantly understand where the risks are, and know what to do about them. Think of it as a first-pass legal radar — fast, clear, and actionable.

3. Core Requirements

3.1 Input Handling

- Accept a PDF upload or pasted plain text as input
- Handle contracts up to approximately 20 pages in length
- Gracefully handle malformed or low-quality input — do not crash

3.2 AI Analysis Pipeline

Your pipeline must do more than a single prompt call. We expect to see deliberate architectural thinking in how you decompose the problem. At minimum, the pipeline should:

- Parse and intelligently chunk the contract to stay within token limits
- Identify and classify clause types (e.g. liability, IP, termination, payment, change control, confidentiality)
- Score each flagged clause for risk severity: High / Medium / Low
- Generate a plain-English explanation of why each clause is risky
- Produce a concrete recommendation for each risk (e.g. 'Negotiate a mutual liability cap', 'Request a 30-day cure period before termination')
- Produce an overall contract risk summary at the top level

3.3 User Interface

The UI is not an afterthought — it is half the challenge. Users of this tool are busy professionals, not developers. Your interface should feel like a product, not a script output.

- Clean, professional design that conveys trust and authority
- A clear risk overview / dashboard view with severity indicators
- Drill-down per clause: original text, risk explanation, recommendation
- Visual hierarchy that lets users scan quickly before reading in depth
- Appropriate loading and progress states — AI calls take time; handle this gracefully
- Responsive layout (desktop-first is fine, but should not break on smaller screens)

3.4 Technical Constraints

- Use any AI API of your choice (OpenAI, Anthropic, Gemini, etc.) — but you must use one
- Frontend framework: your choice (React, Vue, plain HTML — we have no preference)
- Backend: your choice, or serverless — whatever you can deploy
- Must be deployable and accessible via a public URL for review
- No mock data in the submitted version — the AI pipeline must be live

4. What We Are Not Looking For

> ⚠ **Important**
> This challenge is intentionally resistant to vibe coding. A submission that wraps a single prompt in a form and dumps the JSON response to the screen will score poorly regardless of how quickly it was built. We are looking for genuine product and engineering thinking.

Specifically, we are not looking for:

- A single mega-prompt that does everything in one shot
- A JSON dump or raw AI output presented as a UI
- A pre-populated demo with hardcoded example results
- A solution that works only on one specific test document
- Overengineered infrastructure — this is a 24-hour challenge, not a production system

5. Assessment Criteria

Your submission will be reviewed across four dimensions. Each reflects a capability that matters directly to the role at Leveld.

Criterion	Weight	What we're looking for
AI Pipeline Architecture	**30%**	Did you design a deliberate, multi-step pipeline? Are chunking, classification, scoring and summarisation handled as distinct stages? Is prompt design thoughtful?
UX & Interface Design	**30%**	Does the interface feel trustworthy and professional? Is information hierarchy clear? Are loading states handled? Would a non-technical user feel confident using this?
Code Quality & Structure	**20%**	Is the codebase readable and well-organised? Are edge cases handled? Would a colleague be able to navigate and extend this?
Robustness & Accuracy	**20%**	Does it work on documents it hasn't seen before? Are risk identifications genuinely useful rather than generic? Does it handle poor-quality input gracefully?

6. Submission Instructions

1. Deploy your working application to a publicly accessible URL (Vercel, Netlify, Railway, Render — anything works).
2. Share your codebase via a GitHub repository (public or invite-only — share access if private).

3. Include a brief README (max one page) covering: your architectural decisions, which AI model/provider you used and why, any known limitations, and what you would improve with more time.
4. Email your submission to the hiring contact with the subject line: Leveld Challenge — [Your Name].
5. Submissions must be received within 24 hours of you confirming you have started the challenge.

A note on tooling

You are welcome to use AI coding assistants (Copilot, Cursor, etc.). What we're assessing is your judgment, architectural thinking, and product instinct, not whether you typed every line. Your README should briefly note which tools you used.

We have provided a sample contract for you to use as your primary test document during development. It is attached to this brief as a separate PDF file:

- thorngate_msa_draft.pdf — Thorngate Partners LLP, Master Services Agreement, Draft v0.7 (9 pages)

Your submission must be tested against and demonstrably work with this document. It has been designed to stress-test real pipeline decisions — including inconsistent heading styles, mid-clause page breaks, an annex with a resetting page counter, inline amendment annotations, and clauses that require contextual reasoning to score correctly. A naive implementation will produce poor results on it.

We will also test your submission against a second contract document that you will not have seen before. This is to ensure your pipeline generalises rather than being tuned to a single document. Submissions that only work reliably on the provided sample will score poorly on the robustness dimension.

We recommend also testing with at least one additional document of your own choosing before submitting — a different contract type will reveal edge cases your pipeline may not yet handle.

8. Tips from the Team

These are not hints — they're things we genuinely care about:

- Think about the first 10 seconds of the user experience. What does the page look like before they've uploaded anything?
- Severity scoring is easy to get wrong. A clause that limits liability to £1,000 on a £500,000 project is High risk. A standard confidentiality clause is Low. Think about context, not just keywords.

- Chunk deliberately. Splitting naively at 1,000 tokens may cut a clause mid-sentence and produce garbage output. Consider sentence boundaries and clause structure.
- The recommendation is the hardest part. 'This clause is risky' is not useful. 'Request an amendment to cap liability at 200% of contract value' is useful.
- Don't over-engineer the infrastructure. A working app on a free tier is far better than half an architecture that doesn't run.

Good luck — we look forward to seeing what you build.

leveld.ai